Studio Nx

JS 290 — 재생 세션

2026-10-19

JavaScript 290 — 재생 세션 (`audio/session.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/session.js`
작업	S1-11a 재생 오디오 그래프: 엔진, 믹서, 클립, 노트, 이펙트를 묶어 재생, 일시정지, 정지를 소리로 바꿈 (FR-201, FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 289 클립 재생, 다음 291 Synth

1. 하는 일

`PlaybackSession`은 소리 쪽 부품을 모두 모아 "재생 버튼 하나"로 동작하게 하는 지휘자입니다. 페이지의 값(트랙, 클립, BPM, 루프)은 `deps`로 주입받기 때문에 화면 없이도 시험할 수 있습니다.

```
PlaybackSession
  ├ AudioEngine      시계와 스케줄러(lookahead), 메트로놈
  ├ MixerGraph       트랙 믹서와 마스터 (288)
  ├ ClipPlayer       오디오 클립 재생 (289)
  ├ InstrumentRack   MIDI 노트를 악기로 연주 (292)
  ├ NotePlayer       노트 예약 (292)
  └ EffectRack       이펙트 체인 (294)

start(fromTick) ─▶ 샘플을 모두 내려받아 디코딩 ─▶ 믹서와 이펙트를 맞춤 ─▶ 엔진 시작
스케줄러가 1/16박마다 _onStep ─▶ 이 구간에 시작하는 클립과 노트를 예약
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-audio.js` (296)	재생 버튼의 `onPlay`가 `start`를, `onPause`가 `pause`를 부릅니다.
믹서 갱신 타이머	재생 중 150ms마다 `refreshMixer`를 불러 볼륨, 뮤트, 솔로, 이펙트 변경을 소리에 반영합니다.
미터	`meterLevels`로 트랙의 소리 크기를 읽습니다.

3. 코드

3.1 처음 만들기

```
_ensure() {
  if (this.engine) return;
  const self = this;
  const deps = this.deps;
  this.engine = new AudioEngine({
    context: deps.context,
    destination: deps.destination,
    baseBpm: deps.getBpm(),
    changes: deps.getTempoChanges ? deps.getTempoChanges() : [],
```

```
    metronomeEnabled: deps.getMetronome ? deps.getMetronome() : false,
    onStep: function (tick, time) { self._onStep(tick, time); },
  });
  this.mixer = new MixerGraph(this.engine.context, this.engine.destination);
  this.player = new ClipPlayer(
    this.engine.context,
    function (trackId) { return self.mixer.trackInput(trackId); },
    function (sampleId) { return self.buffers[String(sampleId)] || null; },
  );
  this.effects = new EffectRack(
    this.engine.context,
    this.mixer,
    deps.getEffects ? deps.getEffects : function () { return []; },
    function () { return deps.getBpm(); },
  );
  this.rack = new InstrumentRack(
    this.engine.context,
    function (trackId) { return self.mixer.trackInput(trackId); },
    deps.getInstruments ? deps.getInstruments : function () { return []; },
  );
  this.notePlayer = new NotePlayer(this.engine.context, function (trackId, pitch, velocity, when, dur) {
    return self.rack.playNote(trackId, pitch, velocity, when, dur);
  });
}
```

3.2 샘플 불러오기

```
_loadSample(sampleId) {
  const key = String(sampleId);
  if (this.buffers[key]) return Promise.resolve(this.buffers[key]);
  if (this.loading[key]) return this.loading[key];
  const self = this;
  const ctx = this.engine.context;
  this.loading[key] = this.deps.loadSampleBytes(sampleId).then(function (bytes) {
    return ctx.decodeAudioData(bytes);
  }).then(function (buffer) {
    self.buffers[key] = buffer;
    return buffer;
  }).catch(function (err) {
    delete self.loading[key];
    if (typeof console !== 'undefined') console.warn('[audio] 샘플 ' + key + ' 을(를) 불러오지 못했습니
다', err);
    return null;
  });
  return this.loading[key];
}
```

3.3 재생 시작

```
async start(fromTick) {
  this._ensure();
  const self = this;
  const deps = this.deps;
  this.clips = deps.getClips();
  this.noteEvents = expandNotes(deps.getMidiClips ? deps.getMidiClips() : []);
```

```
  await Promise.all(this.clips.filter(function (c) { return c.sample_id != null; })
    .map(function (c) { return self._loadSample(c.sample_id); }));

  this.mixer.applyTracks(deps.getTracks());
  this.effects.sync(deps.getTracks());
  this.engine.setTempo(deps.getBpm(), deps.getTempoChanges ? deps.getTempoChanges() : []);
  if (deps.getMetronome) this.engine.setMetronome(deps.getMetronome());
  const loop = deps.getLoop ? deps.getLoop() : null;
  this.engine.setLoop(loop ? loop.start : 0, loop ? loop.end : 0, !!loop);

  this.player.stopAll();
  this.notePlayer.stopAll();
  this.lastTick = null;
  this.engine.start({ fromTick: fromTick });
  this.playing = true;
  // 시작 위치에 걸쳐 있는 클립은 중간부터 바로 울림
  const t = this.engine.transport;
  clipsCovering(this.clips, t.originTick).forEach(function (c) {
    self.player.schedule(c, t.originTick, t.originTime, self.engine.transport.baseBpm, self.engine.trans
port.changes);
  });
}
```

3.4 한 단계마다 예약

```
_onStep(tick, time) {
  const t = this.engine.transport;
  if (this.lastTick != null && tick < this.lastTick) {
    // 루프로 되감김: 이전 구간 소리는 이 시각에 끊고, 되감긴 위치에 걸친 클립을 다시 시작
    this.player.stopAll(time);
    this.notePlayer.stopAll(time);
    const self = this;
    clipsCovering(this.clips, tick).forEach(function (c) {
      self.player.schedule(c, tick, time, t.baseBpm, t.changes);
    });
  }
  this.lastTick = tick;
  const self2 = this;
  clipsStartingIn(this.clips, tick, tick + t.stepTicks).forEach(function (c) {
    self2.player.schedule(c, tick, time, t.baseBpm, t.changes);
  });
  notesStartingIn(this.noteEvents, tick, tick + t.stepTicks).forEach(function (ev) {
    self2.notePlayer.schedule(ev, tick, time, t.baseBpm, t.changes);
  });
}
```

3.5 재생 중 갱신, 미터, 정지

```
refreshMixer() {
  if (!this.mixer) return;
  this.mixer.applyTracks(this.deps.getTracks());
  this.effects.sync(this.deps.getTracks());
}
```

```
meterLevels(trackId) {
  if (!this.mixer) return null;
  const tracks = this.deps.getTracks();
  const track = tracks.filter(function (t) { return String(t.track_id) === String(trackId); })[0];
  return this.mixer.meterFor(track && track.track_type === 'MASTER' ? 'master' : trackId).read();
}
```

```
pause() {
  if (!this.engine) return;
  this.engine.pause();
  this.player.stopAll();
  this.notePlayer.stopAll();
  this.playing = false;
}
```

```
stop() {
  if (!this.engine) return;
  this.engine.stop();
  this.player.stopAll();
  this.notePlayer.stopAll();
  this.effects.flush();
  this.playing = false;
}
```

4. 최신 문법으로 쓰면

```
// _loadSample 을 async/await 로 쓴 모양
async _loadSample(sampleId) {
  const key = String(sampleId);
  if (this.buffers[key]) return this.buffers[key];
  try {
    const bytes = await this.deps.loadSampleBytes(sampleId);
    return (this.buffers[key] = await this.engine.context.decodeAudioData(bytes));
  } catch (err) {
    console.warn(`[audio] 샘플 ${key} 을(를) 불러오지 못했습니다`, err);
    return null;
  }
}
```

전통 문법 (실제 코드)	최신 문법
`.then(...).catch(...)` 연결	`async/await` 와 `try/catch`
`'[audio] 샘플 ' + key + ' 을(를)'`	템플릿 문자열 `` `...${key}...` ``
`function (c) { return c.sample_id != null; }`	`(c) => c.sample_id != null`
`deps.getMidiClips ? deps.getMidiClips() : []`	`deps.getMidiClips?.() ?? []`

5. 왜 이렇게

선택	이유
엔진, 믹서를 처음 한 번만 만듭니다 (`_ensure`)	브라우저는 사용자 조작 안에서 만든 `AudioContext`만 소리를 내게 합니다. 재생 버튼을 누를 때 만들면 이 조건을 지킵니다.
재생 전에 샘플을 모두 내려받아 디코딩합니다	재생 중에 받으면 처음 몇 초가 비어서 들립니다. 그래서 `start`가 `async`이고, 모든 샘플이 준비된 뒤에 엔진을 시작합니다.
같은 샘플은 한 번만 디코딩합니다	번호별로 버퍼와 진행 중인 약속(`loading`)을 담아 두어서 같은 클립이 여럿이어도 한 번만 받습니다.
재생 시작 시점의 클립을 스냅샷으로 씁니다	재생 중에 클립을 옮겨도 이미 예약한 소리와 어긋나지 않습니다. 옮긴 결과는 다음 재생부터 들립니다.
루프 되감기를 `tick < lastTick`으로 감지합니다	스케줄러가 루프 끝에서 되감으면 tick이 줄어듭니다. 이때 이전 구간의 소리를 되감기 시각에 끊습니다.
정지에서 이펙트를 `flush`합니다	딜레이와 리버브의 꼬리가 다음 재생에 섞이지 않게 합니다.
샘플을 못 받아도 재생은 계속합니다	실패한 샘플만 건너뛰고 경고를 남깁니다 (`null` 반환).

6. 시험

- `docs/handoff/s1-11a_tests/ui.mjs`는 재생, 뮤트, 해제, 솔로, 정지를 마스터 출력의 피크로 확인합니다. 10건 모두 통과했습니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 같은 세션이 MIDI 노트도 재생하는지 확인합니다.
- 시험 중 발견한 일: 예시 화면의 재생선이 12.5마디에서 시작해서 0마디의 클립을 지나쳤고, 1초 샘플은 측정 전에 끝났습니다. 시험이 ■를 눌러 재생선을 0으로 되돌리고 8초 샘플을 쓰도록 고쳤습니다. 소리 코드의 결함이 아니었습니다.

7. 알려진 한계

한계	설명
재생 중 재생선 이동이 소리에 반영되지 않습니다	정지 후 다시 재생해야 합니다.
곡 중간 템포 변경이 소리에 반영되지 않습니다	상단 BPM 값만 씁니다.
노트 중간에서 시작하면 그 노트는 울리지 않습니다	재생 위치 이후에 시작하는 노트만 울립니다.